

ME2 - Voice Controlled Smart Device

ASR models are not desirable for on-device computing because of footprint. The goal of this ME is to build a tiny Voice Command Model (VCM) that can understand the most common commands humans tell their smart devices:

Most common commands issued to smart devices, enumerated

Ranked by real-world log and survey data (259,164 logged commands from Amazon Alexa + Google Home devices, plus recurring U.S. consumer surveys):

Top commands, by rank

1. **Play music** — #1 use case every survey year (2018–2020). Actual forms: *"play music"*
 - For Music tasks (1 and 8), download 5 basic copyright mp3 (like demo songs/alarm) to play randomly when commanded.
2. **Ask a question / search** — *"what's the weather"* (daily-use #2–3), *"what time is it"*,
 - Since the internet is not accessible (Standalone VCM), check locally for the time or date and weather (for that day) in Metro Manila, Philippines.
3. **Control lights (IoT)** — 85% of all Alexa IoT commands: *"turn on/off lights"*,
4. **Dim / color lights** — ~10% of IoT commands: *"Dim lights to X percent"*
5. **Set a timer** — *"set a timer for X minutes"*
6. **Set an alarm** — *"set an alarm for X am/pm"*
7. **Adjust thermostat temperature** — *"set temperature to X degrees"*
 - Use UI simulations for most tasks that require external hardware like lights (3 and 4), clock (5 and 6), and thermostat (7).
8. **Media control** — *"pause", "stop", "next/skip", "volume up/down", "louder"*.
 - No. 1 can be folded to No. 8.
9. **Reminders and lists** — *"remind me to...", "what are my reminders"*
 - Simulate a notes list by printing out these reminders
10. **Calls and messaging** — *"call mom"*
 - Simulate a call (no need to use another app to call)

Your tasks:

1. Build a dataset to train VCMs - collective work
 - One single dataset and schema for everyone
 -  Dataset Schema - OptionB: These are the basis of variations of the commands that everyone will abide to (19 labels/intents for the 10 commands with 3 variations each label and 3 slotted values for some commands, namely: timer, alarm, temperature, brightness, color, and create_reminder)
 - Dataset: <https://huggingface.co/datasets/airimonda/ai231-me2-voice-commands>
 - Documentation File: <https://claude.ai/artifact/PPNHMWd5rx9qcXTdcukV7s>
2. Build and train a VCM on this dataset - individual work
 - Compare atleast 5 models (CRNN, CNN, and others suitable for this)
 - Train for 50-100 epochs until no improvements are seen and select best model
 - Best model will be used for benchmarking and demo in RPi4
3. Design a benchmark for validating VCMs - collective work
 - <https://github.com/airimonda/vcm-benchmark>
4. Validate the performance of your VCM - individual work (using own voice)
5. Build a real-world demo of your VCM (this can run on RPi4/5 device) - individual work
6. Your VCM must be tiny - can run on RPi4 or 5 in real-time - individual work
7. VCM should be standalone - can not call cloud-based models.
8. No LLM, just pure VCM doing 1 to 10. Everything on-device.
9. Ok to use a wake word (e.g. Alexa, Google, etc)
 - Include a open source, pre-trained lightweight, tiny model for the wakeword that can operate in RPi4 4GB

ME2 — Voice Controlled Smart Device

Always-on keyword + intent recognition at the edge · no cloud round-trip

1 October 2026 · <fill: author / team> · Model · Dataset · Training on A100 · Validation on RPi 4

Model

Mic · 16 kHz → log-mel (TBD) × (TBD)	
Causal encoder × (TBD) · (TBD)-dim KV-cache · streaming	
Intent head	Slot head
Actuator	Raspberry Pi 4
Item	Value
Parameters / weights	(TBD) M · (TBD) MB

Validation on the Raspberry Pi 4

Item	Value
Keyword / intent acc	<TBD> % / <TBD> %
False-accept rate	<TBD> %
Latency p95 / RTF	<TBD> ms / <TBD>
Runtime	onnxruntime · <TBD> thr

Dataset

Item	Value
Source	<fill>
Hours / utts	<TBD> h / <TBD>
Speakers	<TBD>
Labels	<TBD> intents · <TBD> slots

Training on the A100 cluster

Item	Value
Cluster	<fill: # A100>
Objective	<fill: CTC / attn>
Optimiser	<fill>
Steps / loss	<TBD> / <TBD>

p. 1 / 2

* Upon figuring out the best model among the 5 tested model architectures, fill in the Model (architecture) and Dataset values (from the links) but leave the RPi4 and A100 cluster tables blank in the meantime since I am still training the model in my own computer.

To Be Submitted

GitHub repository	<fill: org/repo – public, MIT>
Dataset location	<fill: Zenodo / HF DOI + access terms>
A100 cluster	<fill: node ID, # GPUs, wall-clock, seeds>
Model weights	<fill: release URL + licence>

What the reviewer will look for

#	Item	Status
1	Repo public, one-command reproduction	<todo>
2	Dataset licensed and citable (DOI)	<todo>
3	Training logs + final checkpoint committed	<todo>
4	Pi 4 latency reproduced by the posted script	<todo>
5	Held-out test set, unseen speakers	<todo>
6	Baseline of comparable size compared	<todo>

p. 2 / 2

* Github repository is for reproducibility

* Dataset location and A100 cluster related values will be later when the cluster has space so leave these sections blank since I am currently working locally on my own computer

* The rest, produce accordingly. For the “What the reviewer will look for”, create a document containing these sections. For #1 (Repo), update MLOps Exercises/ME2 in my github with the new files. For #2 and #5, include the collated dataset and the held-out test set from HuggingFace and the Claude Documentation. For #3 and #6, indicate the training logs and final checkpoint committed by the best model and the baseline of comparable size compared. For #4, write the section but leave a template/empty so that I can later on put the necessary information there.

Someone noted:

bale, isesetup nyo yung pi nyo, then ipupull nito ung holdout natin, then it will ask you to record your wakeword thrice, then will patch wakeword + command. para pwede nyo na syang iwan mag isa nagsasalita. tas ang laman ng report.md natin ay:

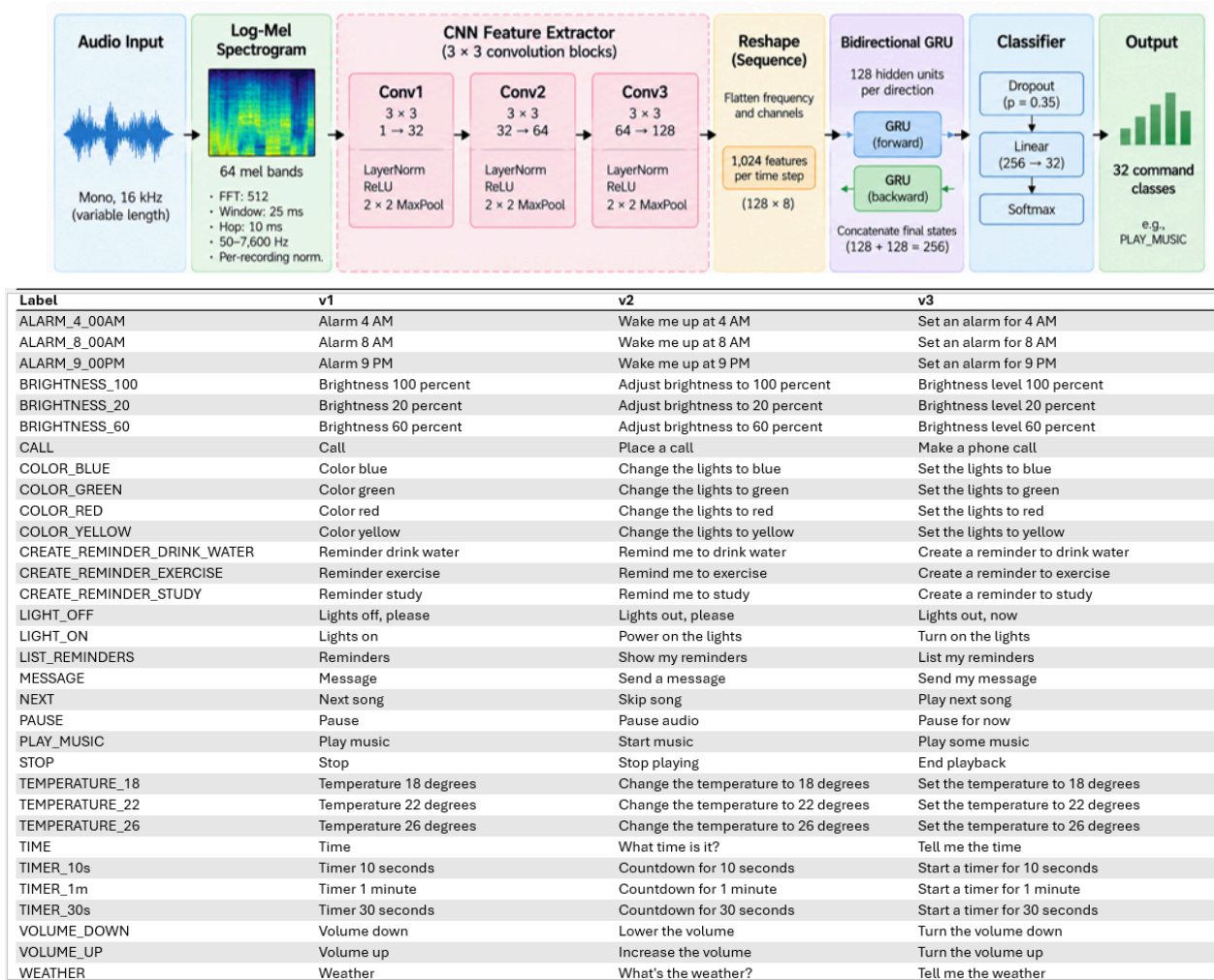
1. **Header:** student, date, wake word, number of commands, connection type, holdout source.
2. **Classification table**, with one column for the 19 intents and one for the 93 commands. Each column shows:
 - accuracy, with its 95% confidence range;
 - balanced accuracy;
 - precision, recall, F1 and F2 (averaged over the classes in the test);
 - false accept rate: out-of-scope clips where the Pi fired a command, with its range and count (e.g. 1/10);
 - false reject rate: real commands where the Pi said "out of scope" or stayed silent;
 - misfire rate: real commands where the Pi fired the wrong command.

Below the table:

- Pipeline: how often the Pi responded at all, no-response count, extra fires (more than one command per clip), and wake detection rate.
 - Unknown intent names: any name the Pi printed that isn't one of the 19 (scored wrong),
 - Real vs synthetic voices: accuracy for each.
3. Slot values: one row per slotted intent, plus an "all" row. It only covers clips where the Pi got the intent right. Columns:
 - exact-match rate;
 - mean absolute error (Manhattan distance) in the slot's unit: seconds, minutes, degrees or percent;
 - mean relative error, so different slots can be compared;
 - phonetic distance and character distance.
 4. Raspberry Pi
 - Specs: model, hostname, cores, CPU, top clock speed, RAM, OS, kernel, Python version, installed ML/audio packages.
 - Metrics table: mean / 95th percentile / max for each:
 - response latency (end of command to Pi output), plus 50th and 99th percentiles;
 - inference time and real-time factor (only if the Pi prints infer_ms and audio_ms);
 - CPU temperature, CPU clock, load average, throttling flags;
 - CPU use and RAM of the whole Pi, and of the student's assistant process;
 - CPU-seconds per second of speech, and the assistant's CPU share of the test time;
 - test duration.
 - Model (if an ONNX model was given): parameter count, size, FLOPs per inference, and effective GFLOP/s.
 5. Most frequent confusions: the top 10 "expected → got" pairs at intent level and at command level.
 6. Per-intent scores: for each intent (and REJECT), the number of clips, precision, recall, F1 and F2.

Another shared their architecture and labels:

CRNN Architecture



- Create a similar model architecture figure and class labels with variations within the dataset
- Produce Figures for some evaluation metrics (such as, but not limited to: per intent recall matrix, training curves of the 5 models for easier comparison of best model, etc.)